

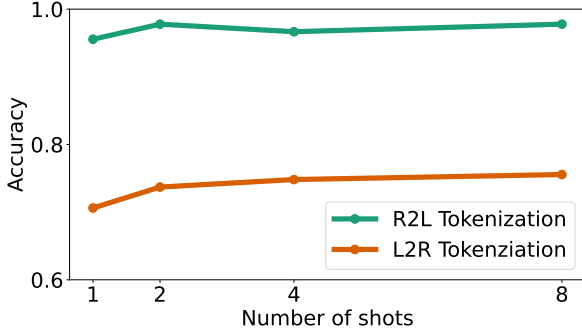


Figure 4. Effect of R2L vs L2R tokenization with increasing shots.

depicted in Figure 1, so the observed effect may be confounded by prevalence in training data (McCoy et al., 2023). One might argue that comma separation is actually bringing the input closer to the training distribution of the model, so it’s not a surprise that models perform better. To control for this and focus in on tokenization, we consider alternate, single-token separators: ‘ ’, ‘.’, ‘\$’, ‘#’ (note we’ll refer to ‘ ’ as `<space>` for clarity). For example, the number 8302080 would be written as 8#302#080 when input to the model.

Results are shown in Figure 5. We find that the model is largely agnostic to the separator used, indicating that tokenization is likely the dominant effect, rather than the specific choice of using commas.

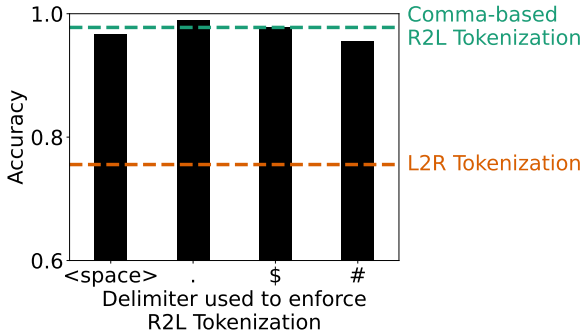


Figure 5. 8-shot accuracy when using different delimiters for R2L tokenization. Dotted lines show results from Figure 1 for comparison. Overall, we see choice of delimiter matters less than direction of tokenization.

3.3. Controlling for “thinking tokens”

Another confound with the above experiment may be that adding commas both increases the number of tokens input to as well as generated by the model. Thus, to generate the same answer, the model has access to more computation steps (i.e., FLOPs). There is a worry that models may use these repetitive *thinking tokens* to perform additional useful computations (Lanham et al., 2023). In practice, this seems not to happen without further training (Goyal et al., 2024), but we conducted experiments to verify this in our setting.

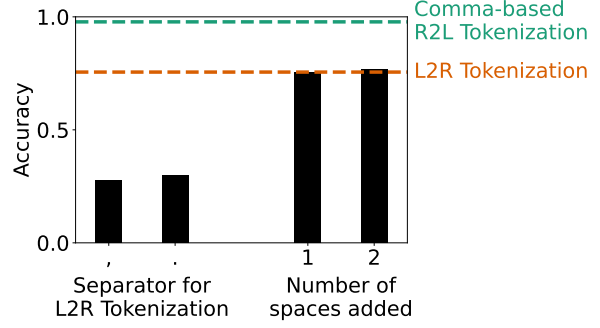


Figure 6. 8-shot accuracy for various “thinking token” controls. Dotted lines show results from Figure 1 for comparison. We also experimented with other delimiters for L2R tokenization (all those from Figure 5), but found similarly poor results. Overall, “thinking tokens” do not recover the performance boost from using comma-enforced R2L tokenization.

To control for thinking tokens, we consider two types of controls. In the first, we use separators to enforce L2R tokenization – this enforces an exact match in prompt token counts. Second, we consider adding 1 or 2 spaces before and after the + and = sign to increase the number of tokens⁷ in the L2R case (where no separator is used). Both of these have the benefit of adding extremely “predictable” tokens (when using 8-shots), allowing the model to possibly use the extra computation steps for “thinking”.

In Figure 6, we find that neither of these controls, when applied to L2R tokenized sequences, recovers the performance of R2L tokenization. In fact, we found that using separators with the L2R tokenization often hurt performance, likely because this is an uncommon representation—upon qualitative inspection of a few examples, we found the model sometimes “auto-corrects” the inputs by hallucinating trailing zeros. We believe these experiments effectively rule out the “thinking token” confound.

4. Error analysis reveals stereotyped patterns

Given the robust effect observed in Section 3, we were curious to see if there were any patterns in the errors. Below, we summarize our key findings.

4.1. L2R tokenization is significantly worse when answer is longer than addends

As noted in Section 2.1, we balanced our dataset of problems based on input digit length. Upon inspection of problems the model got incorrect when using L2R tokenization, we noted that errors seemed more likely when the answer was longer than the addends (e.g., a problem where 7 digit number + 7 digit number = 8 digit number, of the form depicted

⁷Specifically, the token count in the 8-shot prompt increases from 195 to 213 to 240 when going from 0 to 1 to 2 spaces. For reference, the R2L token count is 247.

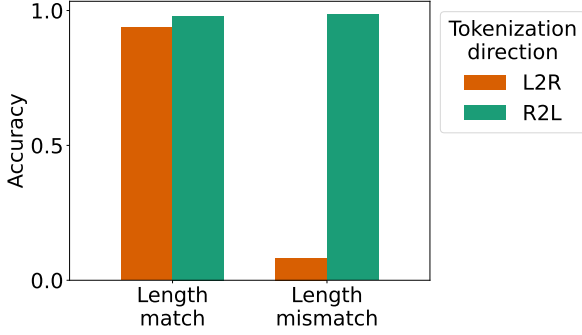


Figure 7. When the answer is the same length in digits as an addend (length match), both tokenization schemes perform similarly (left). When the answer is a different length in digits than either addend (length mismatch), L2R tokenization destroys model performance, dropping to 8.25% (right).

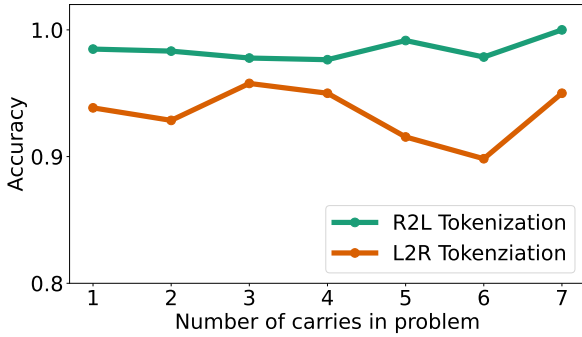


Figure 8. Accuracy as a function of number of carries. For L2R tokenization, we exclude problems where the answer length does not match at least one addend, as the model misses most of those (92%) as shown in Section 4.1. If number of carries (a human notion of difficulty) was correlated to model performance, we would expect a negative slope. The lack of any trend suggests model performance is largely independent of number of carries.

in Figure 1). To test this hypothesis, we conducted a new experiment where we controlled for addend lengths *and* answer lengths. Specifically, we generated 100 random problems for each possible triplet of digit lengths where addends and answer have a length of 7 to 9 digits (full list in Appendix E.1). The remainder of our experiments in this section will use this expanded set of problems to show the robustness of the found error patterns.

We reproduced our main phenomenon (Section 3.1), and further affirmed our intuitions about error patterns. As shown in Figure 7, we find that L2R tokenization has similar performance to R2L tokenization when the answer’s length in digits is the same as one of the inputs (which we refer to as the “length match” condition). When the answer is longer than the inputs (due to a final carry), L2R tokenization is significantly worse, with accuracy dropping down to 8.25% – we refer to this as the “length mismatch” condition. We suspect that this strong effect may be due to the misalignment between input and output tokenizations (as illustrated in

Figure 1) rather than some carry-related notion of problem difficulty, which we explore in the next few subsections.

4.2. Errors do not seem correlated to number of carries

A natural hypothesis given the above result may be that errors might just be correlated to some notion of difficulty, such as carries. In Figure 8, we find that this is generally not the case. Specifically, we consider the accuracy on subsets of problems based on how many carries are needed to solve them.⁸ The lack of a clear positive or negative trend indicates that model performance is not strongly affected by the number of carries.

4.3. Length mismatch problems yield stereotyped “digit 4” error pattern

If not carries, what could be causing the surprising error pattern in Figure 7? In Figure 9a, we find that the errors when using L2R tokenization are extremely stereotyped and not at all intuitive. Specifically, in the length mismatch condition, the model *always* gets the fourth digit wrong. Furthermore, the model *always* gets the first 3 digits correct (corresponding to the first output token). In terms of how far off the model is on digit 4, Figure 9b shows that there’s a slight preference to off-by-one errors, but overall the specific substitution appears quite haphazard.

We found this result extremely surprising. In cognitive science, such stereotyped error patterns are often used as evidence of underlying systematic processing. While the mechanism for addition in LLMs remains unclear, we find this striking, tokenization-dependent error pattern⁹ as highly suggestive of some underlying algorithm (in contrast to suggestions that LLMs may be performing arithmetic using some “fuzzy” matching to similar problems in training). We provide further evidence of stereotyped error patterns by analyzing model log probabilities in Appendix D.

4.4. Off-by-one errors at token boundaries account for nearly all remaining errors

After accounting for the main source of error, we analyzed the remaining errors across both tokenization methods: 25 out of the 1300 problems for R2L tokenization, and 56 out of the 900 problems in the length match condition for L2R tokenization.

For R2L tokenization, of the 25 problems missed, 24 are due to off-by-one (either above or below) errors. For L2R tokenization, of the 56 problems missed, 53 are due to off-

⁸Since we didn’t explicitly control for carries when generating problems, the number of problems with a given number of carries varies. We only considered cases where we had at least 50 problems.

⁹This error pattern is not present when using R2L tokenization.

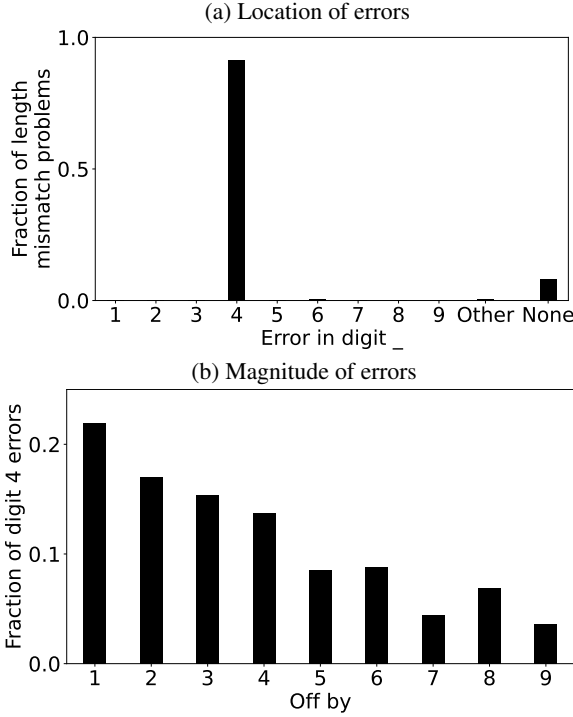


Figure 9. **a)** Error patterns for L2R tokenization over problems where the answer digit length is different than the addend lengths. “None” indicates the problems in this case that the model got correct (8.25%). “Other” indicates problems where the model doesn’t provide a valid answer or provides an answer of the wrong length (0.5%). Of the remaining 91.25% which the model gets incorrect, it shockingly *always* gets digit 4 wrong. In addition, it sometimes gets other digits (5, 6 or 7) wrong. **b)** For the errors in digit 4, we show the magnitude of the mistake. For example, if the correct value of digit 4 is 2 and the model response has digit 4 equal to 5, it would be off by 3. We see a slight preference to off-by-1 errors, but error magnitudes are fairly evenly distributed.

by-one (either above or below) errors. For nearly all these off-by-one errors,¹⁰ regardless of tokenization direction, we find that the error itself occurs in *the last digit of an output token*. This result suggests that off-by-one errors are more likely across token boundaries as opposed to in the middle of a 3-digit token. This hypothesis, with preliminary evidence, connects to works on length generalization (Anil et al., 2022) – using 3-digit tokens may make length generalization easier as models only need to cross token boundaries every third digit (as opposed to every digit).

5. Models are able to convert from L2R to R2L tokenization, improving performance

5.1. Main results

With the above results showing that number tokenization can strongly affect numerical reasoning, we ask if mod-

¹⁰Specifically, all 24 off-by-one errors in the R2L case, and 51 of the 53 off-by-one errors in the L2R case.

els can be prompted to take problems in a less preferred tokenization (L2R) and convert them to a more preferred tokenization (R2L) to improve performance. Inspired by chain-of-thought approaches (Nye et al., 2021; Kojima et al., 2022; Wei et al., 2022), we few-shot prompt models to take problems with one tokenization direction, and then repeat the problem and answer it using a different tokenization direction. In Figure 10, we find that models indeed perform nearly as well at addition when converting L2R tokenization to R2L themselves as to when they receive the problem in R2L tokenization in the first place. Performance increases with the number of shots when converting L2R to R2L since the model adheres more to the (helpful) suggested repetition style. These results indicate that models can convert between tokenizations to solve problems correctly, but do not do so implicitly in the forward pass.

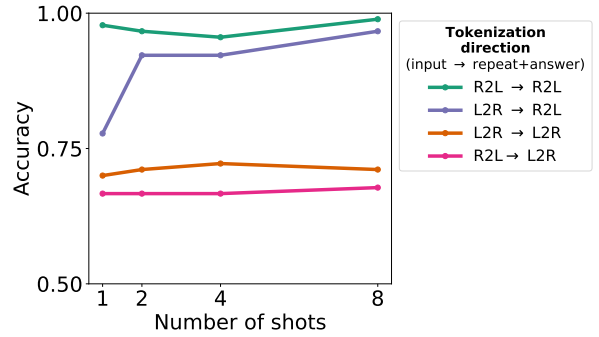


Figure 10. Few-shot accuracy when models receive a problem with one tokenization direction, then repeat and answer it in another.

5.2. Controlling for output tokenization

One confound with the above experiment may just be that the model improves when it’s asked to generate *answers* with R2L tokenization. To control for this, we conduct a similar experiment, but without few-shot prompting the model to repeat the problem: the few-shot prompt provides answers with a different tokenization direction than the input, incentivizing the model to answer with this tokenization direction (see Appendix E.3 for an example prompt). In Figure 11, we see that just answering with R2L tokenization does not improve performance (purple curve) to the degree that repeating in R2L tokenization does (purple curve, Figure 10), when starting from L2R tokenization. This effect indicates that it is important for the model to also *see* the problem in the preferred tokenization (by repeating it), rather than just answering in the preferred tokenization.

6. Tokenization-dependent effects mostly extend to future models

Through the previous sections, we’ve demonstrated a strong tokenization-dependent effect. In this section, we address the question: does this effect extend to newer models?